

Validation 3 — Semi-Analytical Kernel & RL Pricer

We replace the single-sample TD bootstrap in D4PG with a **deterministic quadrature target**:

$$Q_{\text{target}}(s, a) = r(s, a) + \mathbb{E}[Q(s', \pi(s')) \mid s, a],$$

where the expectation is evaluated analytically over the **HHK one-step transition kernel** (Gauss–Hermite on the OU factor $\times$ compound-Poisson jump mesh).

Outline.

- **Part I** — *Is the quadrature correct?* Spectral convergence, nested-MC agreement, cost/accuracy frontier.
- **Part II** — *Does the RL policy price well?* Three pricers compared on one common OOS test set (seed 999, 65 536 paths):
 - **LSM-D** — full-state Chebyshev deg-2, $M \in \{5, 9, 17\}$ exercise levels
 - **RL (no kernel)** — v61 D4PG, single-sample TD, 32 768 ep
 - **RL-kernel** — v64 D4PG, expected backup ($M_x=2$), 4 096 ep

Regimes: no-cost ($c=0, \gamma=1$) and $c=0.04$ at $\gamma \in \{1, 1.5, 2\}$ (focal $\gamma=2$).

Loaded: {'moments': (19, 7), 'quad': (168, 9), 'jump': (44, 12), 'nested': (72, 10), 'cost': (6, 9), 'pricing': (67, 11), 'episodes': (39, 10), 'mx': (27, 8)}

Part I · Mathematics of the HHK transition kernel

Under $S_t = \exp(f(t) + X_t + Y_t)$ with OU diffusion $dX = -\alpha X dt + \sigma dW$ and compound-Poisson jumps $dY = -\beta Y dt + J dN$, the one-step transition factorises into an **exact Gaussian leg** and a **jump leg**. The kernel uses M_x Gauss–Hermite nodes on X' and a stratified-QMC mesh on Y' ; total node count $M = M_x(1 + N_{\max} M_{\text{per } k})$.

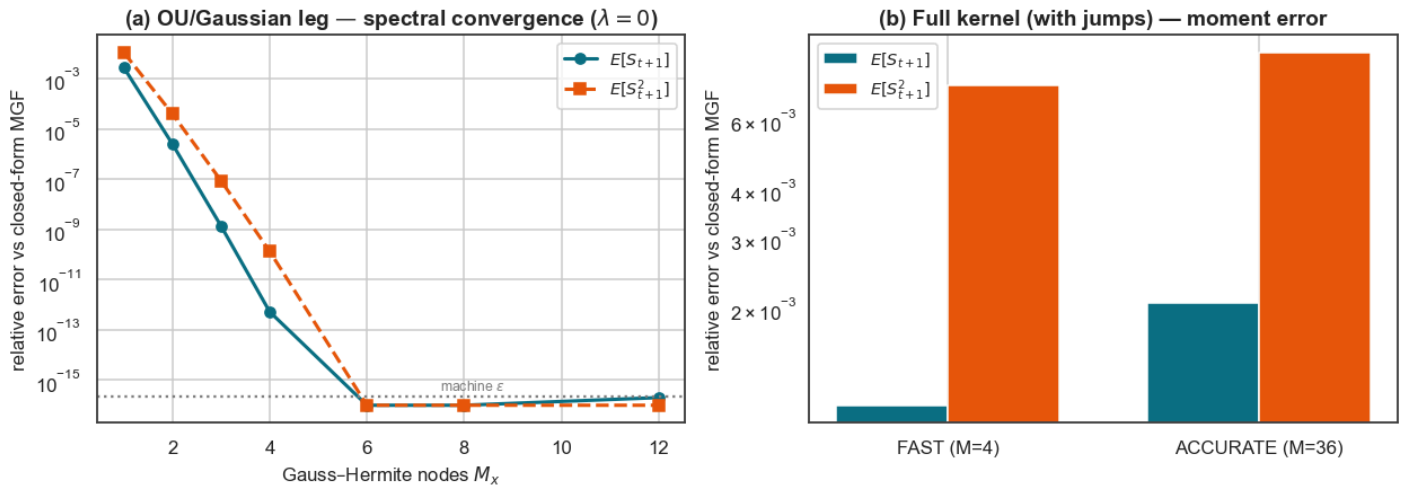
- **FAST** ($M_x=2, M_{\text{per } k}=1, N_{\max}=1$) $\Rightarrow M=4$ — production kernel
- **ACCURATE** ($M_x=4, M_{\text{per } k}=4, N_{\max}=2$) $\Rightarrow M=36$ — convergence reference

How the mesh is chosen

- **OU leg (M_x Gauss–Hermite nodes)**: The integrand is smooth against a Gaussian density $\Rightarrow$ spectral convergence; $M_x=1$ collapses (point mass), $M_x \geq 2$ is fully converged (F1–F2, R5). M_x is the sole controlling axis.
- **Jump leg ($N_{\max}, M_{\text{per } k}$)**: Compound-Poisson sum with $\lambda \Delta t \approx 0.029$, so $\geq 97\%$ mass on $k=0$. $N_{\max}=1-2$ drives the truncation tail below the Gaussian-leg floor; $M_{\text{per } k}$ barely matters once ≥ 1 .

F1 · OU/Gaussian leg — spectral convergence

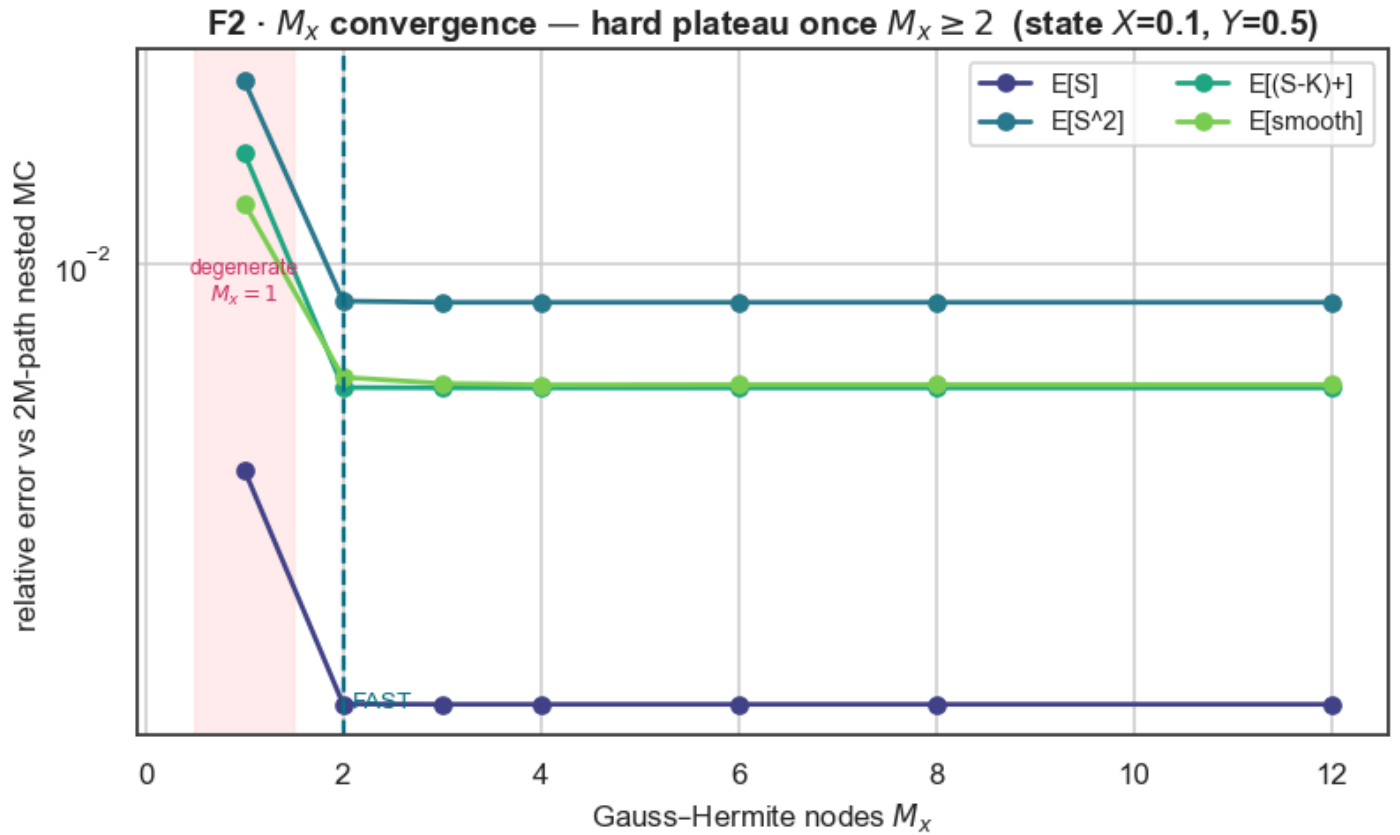
With $\lambda=0$ the transition is purely Gaussian; the conditional moments $\mathbb{E}[S_{t+1} | X_t]$ and $\mathbb{E}[S_{t+1}^2 | X_t]$ have closed forms (OU MGF). Gauss–Hermite quadrature should converge spectrally in M_x .



Gaussian leg reaches machine precision by $M_x=4$; with jumps the error settles at the jump-mesh floor ($\sim 10^{-3}$ on $E[S]$) – the subject of F2/F3.

F2 · Full-kernel convergence in M_x vs nested Monte-Carlo

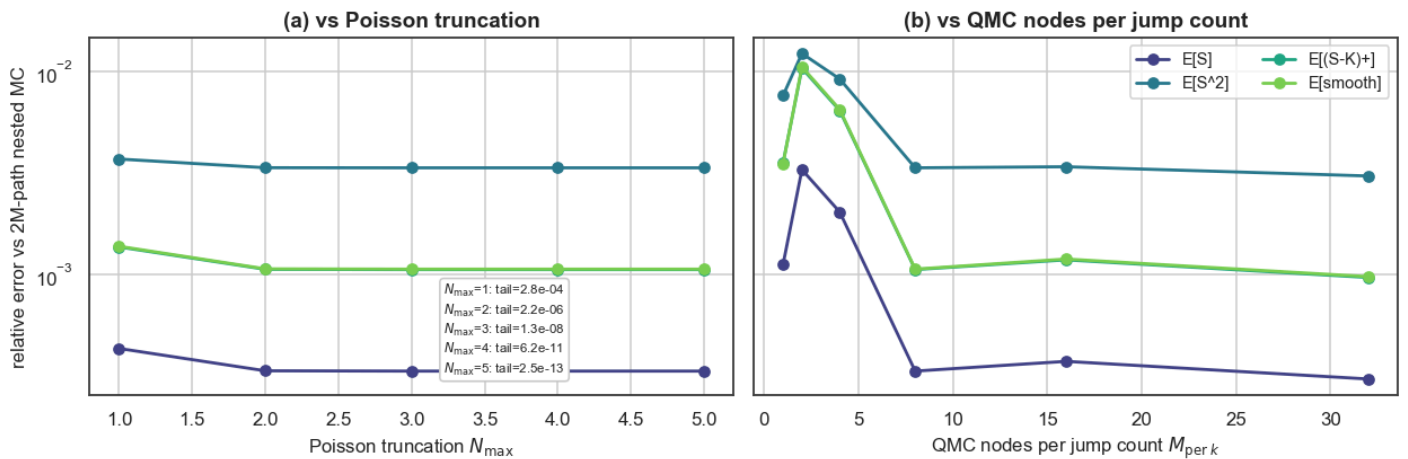
Full kernel expectation vs a 2M-path nested MC reference for $E[S]$, $E[S^2]$, $E[(S-K)^+]$, and a smooth payoff proxy, as M_x increases (jump mesh held at ACCURATE). The curve should plateau at the jump-mesh floor once $M_x \geq 2$.



F3 · Jump-mesh convergence

Residual error from F2 is the jump-mesh discretisation, controlled by Poisson truncation $N_{\max}$ and per-count QMC resolution $M_{\text{per } k}$. Since $\lambda \Delta t \approx 0.029$, the tail beyond $N_{\max}=2$ is negligible.

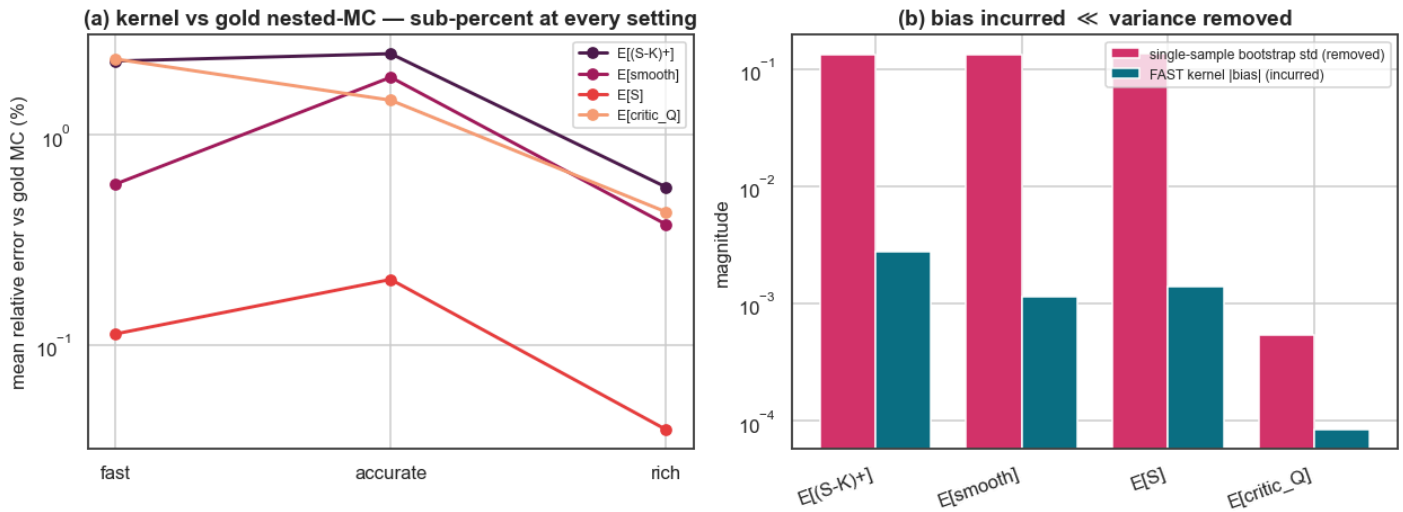
F3 · Jump-mesh convergence



F4 · Agreement with nested Monte-Carlo for fixed state-action pairs

For fixed states we compare the deterministic kernel target against a 4M-path nested-MC gold reference, for payoff integrands and the **actual critic network** $Q(s', \pi(s'))$. **(a)** Kernel bias is sub-

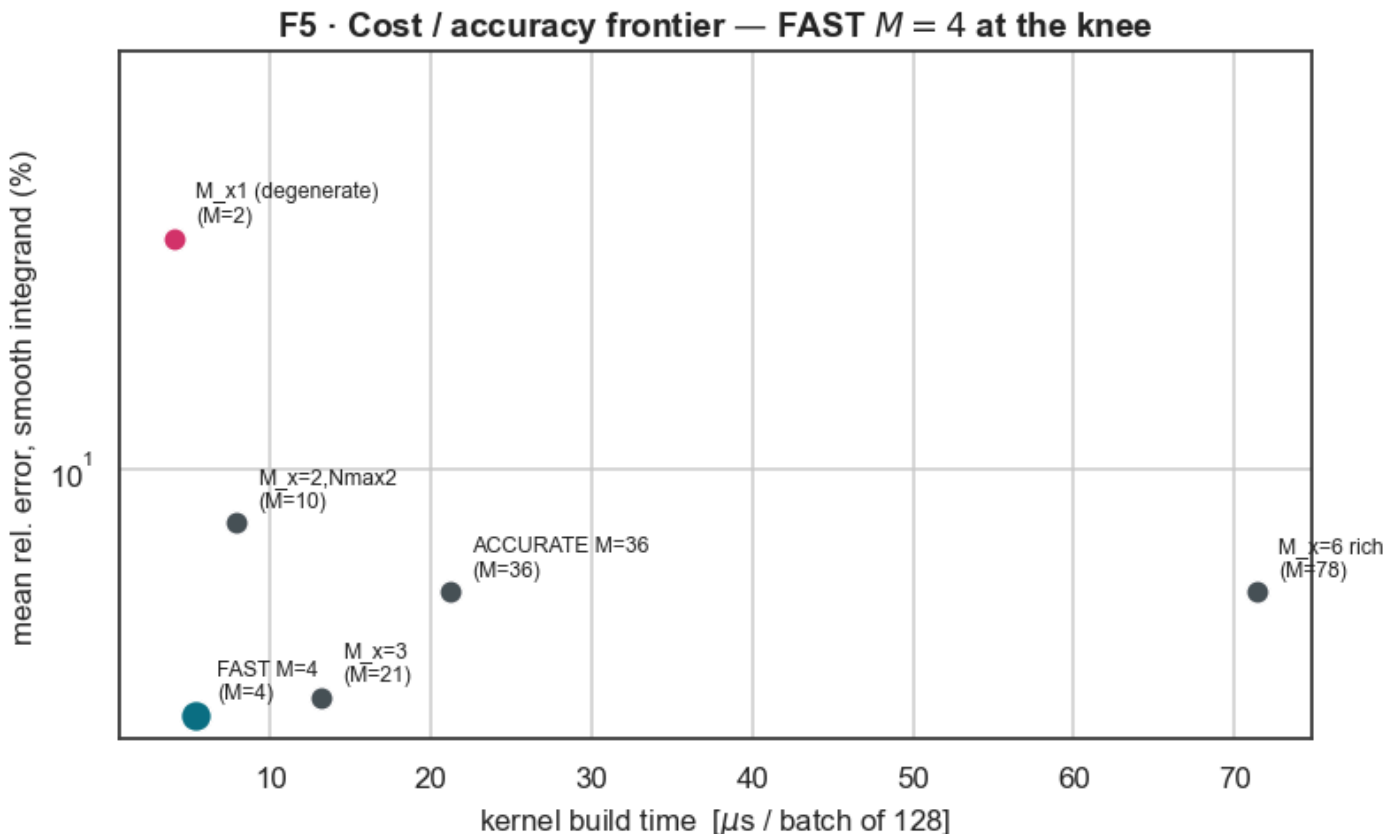
percent at every mesh setting; **(b)** that bias is $6\text{--}120\times$ smaller than the single-sample bootstrap noise it replaces.



FAST kernel: single-sample noise / kernel bias $\approx 6\text{--}118\times$ across integrands (the variance-reduction factor of the expected backup).

F5 · Cost / accuracy frontier

Per-call build time ($B=128$ batch) vs accuracy on the smooth continuation integrand. The FAST $M=4$ kernel sits at the Pareto knee — sub-percent error at a few microseconds.



	label	M	relerr_ES	relerr_smooth	worst_relerr	build_us_b128
0	M_x1 (degenerate)	2	0.00386	0.3058	0.842	4.0
1	FAST M=4	4	0.00115	0.0303	0.600	5.4
2	M_x=2, Nmax2	10	0.00332	0.0771	0.761	7.8
3	M_x=3	21	0.00120	0.0331	0.247	13.2
4	ACCURATE M=36	36	0.00207	0.0551	0.210	21.2
5	M_x=6 rich	78	0.00207	0.0550	0.144	71.4

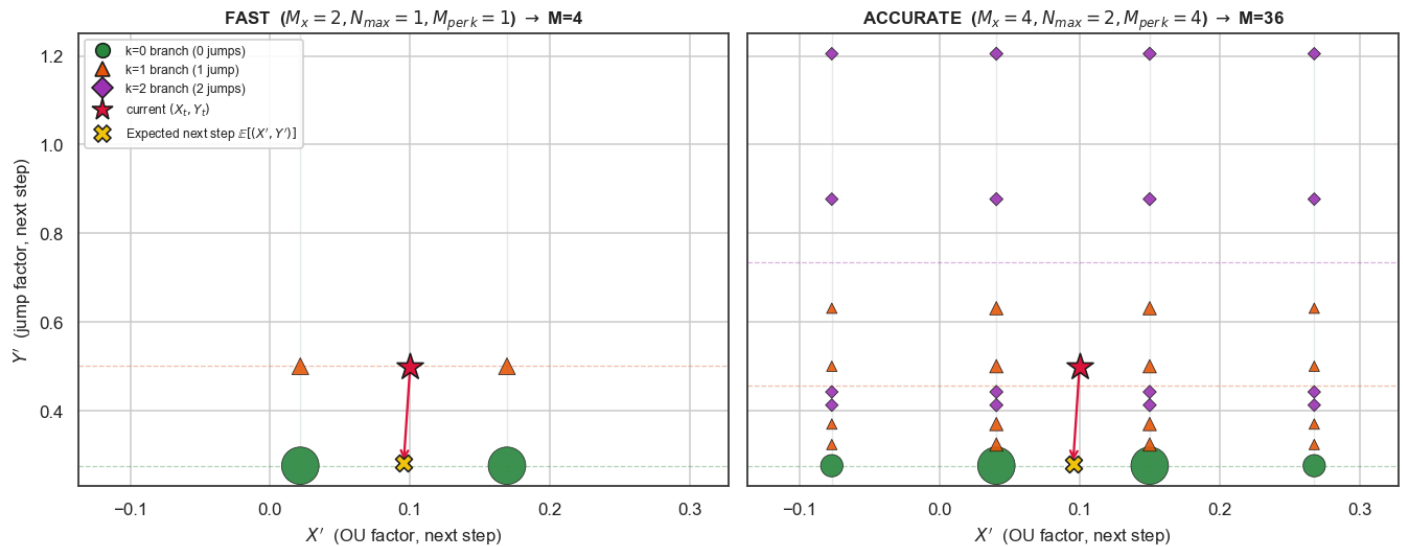
F6 · The quadrature mesh in state space

For a current state (X_t, Y_t) the kernel evaluates the continuation on a **product grid** of next states:

$$X'_i = e^{-\alpha \Delta t} X_t + \sigma_X z_i, \quad Y'_j = e^{-\beta \Delta t} Y_t + \Delta_j, \quad w_{ij} = w_i^X w_j^Y.$$

The M_x columns are Gauss–Hermite nodes (OU shock); the rows are jump-increment nodes — one no-jump node plus $M_{\text{per } k}$ QMC nodes per jump count $k = 1, \dots, N_{\text{max}}$. The no-jump branch carries $\approx 97.6\%$ of the mass.

F6 · The quadrature mesh in next-state (X', Y') space (marker area $\propto$ weight; example $X_t = 0.1, Y_t = 0.5$)



FAST M=4, ACCURATE M=36 | no-jump weight $P(0)=0.9765$, $\lambda \cdot dt=0.0238$ | $\sigma_X(\text{FAST})=0.0738$, $\text{decay}_Y=0.5516$

Part I takeaway. The kernel is mathematically sound: OU leg exact to machine precision, jump mesh converges by $N_{\text{max}}=2$, deterministic target agrees with nested MC to $\lesssim 0.1\%$ — a bias $\sim 100\times$ smaller than the single-sample bootstrap variance it eliminates. FAST $M=4$ is the cost/accuracy optimum.

Part II · Validation of the RL pricer

Every saved policy is evaluated by **forward rollout on one common test set** (seed 999, 65 536 paths).

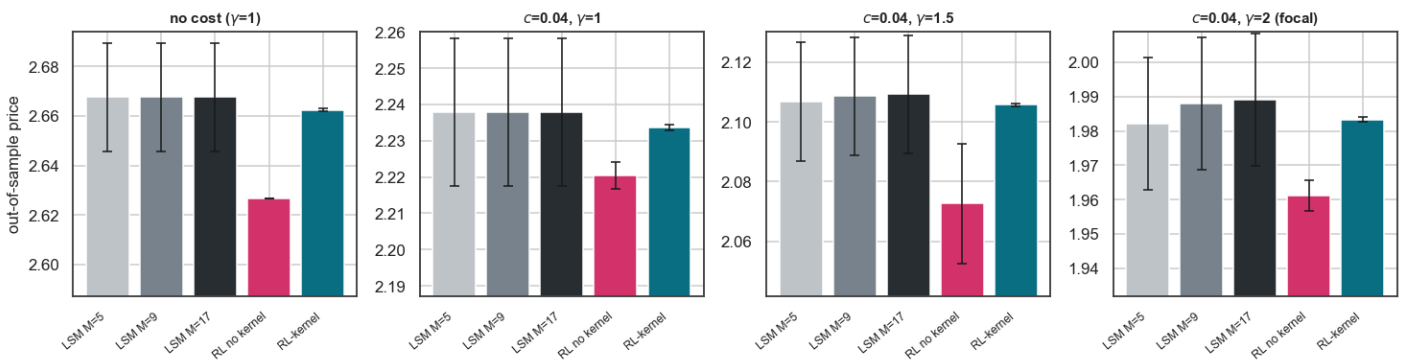
Method	Training target	Budget	Source
LSM-D	backward induction (Chebyshev deg-2, seed=998)	$M \in \{5, 9, 17\}$	refit live
RL (no kernel)	single-sample TD (v61 agents)	32 768 ep	saved prices
RL-kernel	expected backup, $M_x=2$ (v64 recipe)	4 096 ep	evaluated live

The two RL variants use different canonical configs (both defined in `tools/train_hedging_agents.py`).

R1 · Forward-rollout pricing vs strengthened LSM-D

Mean out-of-sample price across seeds for RL-kernel (4k ep), RL no-kernel (v61, 32k ep), and LSM-D at $M \in \{5, 9, 17\}$. $\Delta\% = (V_{\text{RL}}/V_{\text{LSM}} - 1) \times 100$ reported against each M .

R1 · Forward-rollout price — mean $\pm$ 95% CI (MC for LSM; cross-seed SEM for RL)



$\Delta\%$ vs LSM-D (positive = RL above LSM)

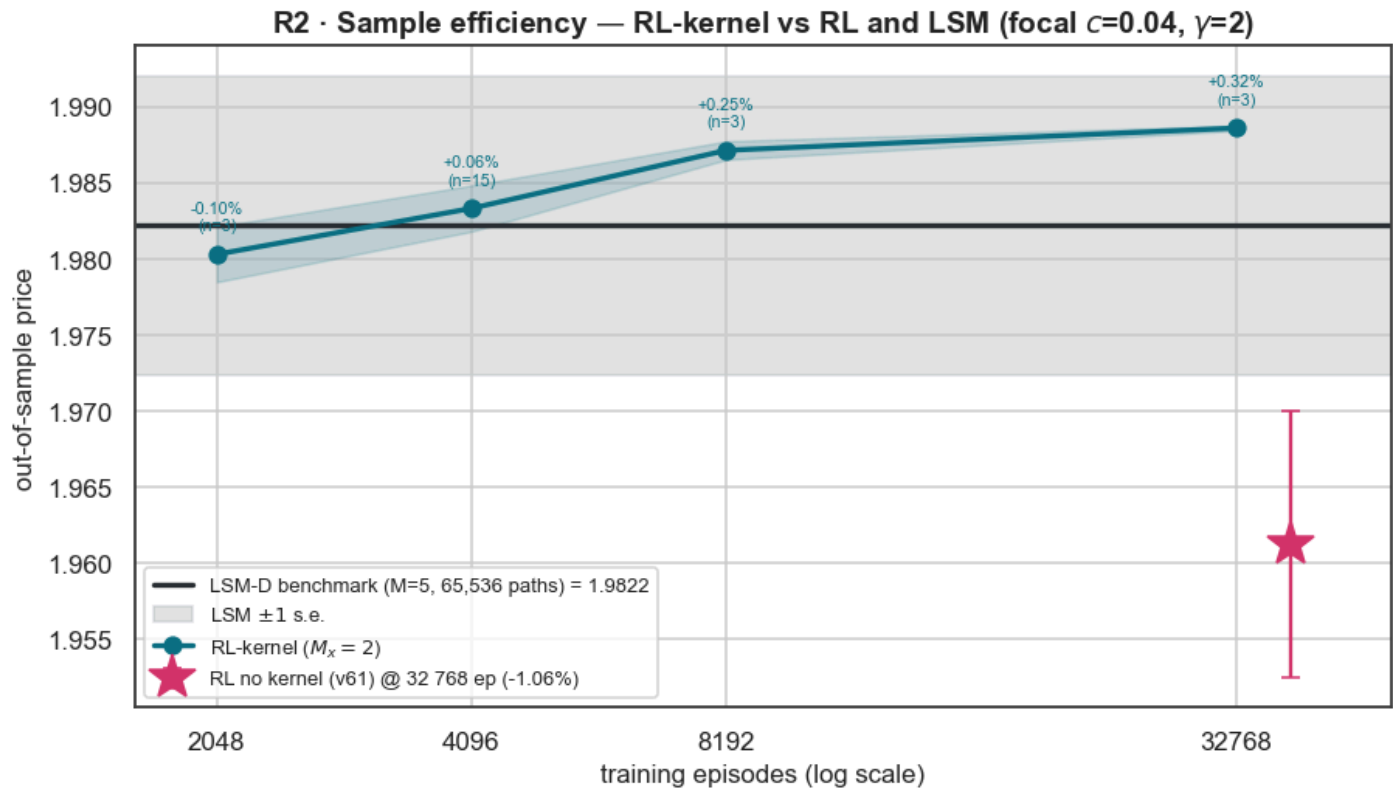
	RL-kernel vs M5	RL-kernel vs M9	RL-kernel vs M17	RL vs M5	RL vs M9	RL vs M17
regime						
no cost ($\gamma=1$)	-0.19	-0.19	-0.19	-1.54	-1.54	-1.54
$c=0.04, \gamma=1$	-0.18	-0.18	-0.18	-0.78	-0.78	-0.78
$c=0.04, \gamma=1.5$	-0.05	-0.14	-0.17	-1.61	-1.70	-1.73
$c=0.04, \gamma=2$ (focal)	+0.06	-0.23	-0.29	-1.06	-1.34	-1.40

Welch test: RL-kernel vs RL no kernel (per-seed OOS price)

	$\Delta\text{price (pp)}$	p-value	n (kernel / no-kernel)
regime			
no cost ($\gamma=1$)	—	—	12 / 1
$c=0.04, \gamma=1$	+0.0134	0.017	3 / 3
$c=0.04, \gamma=1.5$	+0.0329	0.084	3 / 3
$c=0.04, \gamma=2$ (focal)	+0.0221	0.000	15 / 15

R2 · Sample efficiency — RL-kernel vs RL (no kernel)

The deterministic target lets the kernel critic learn the continuation value in far fewer episodes. RL-kernel price tracked against training budget, with LSM $M=5$ (solid) and RL no-kernel @ 32k ep (star) as references.



RL-kernel @32768 ep: +0.32% of LSM (n=3).

LSM-D benchmark: $M=5$, fit on 65,536 train paths (seed 998), priced on 65,536 test paths (seed 999).

RL no kernel (v61) @32 768 ep: -1.06% of LSM.

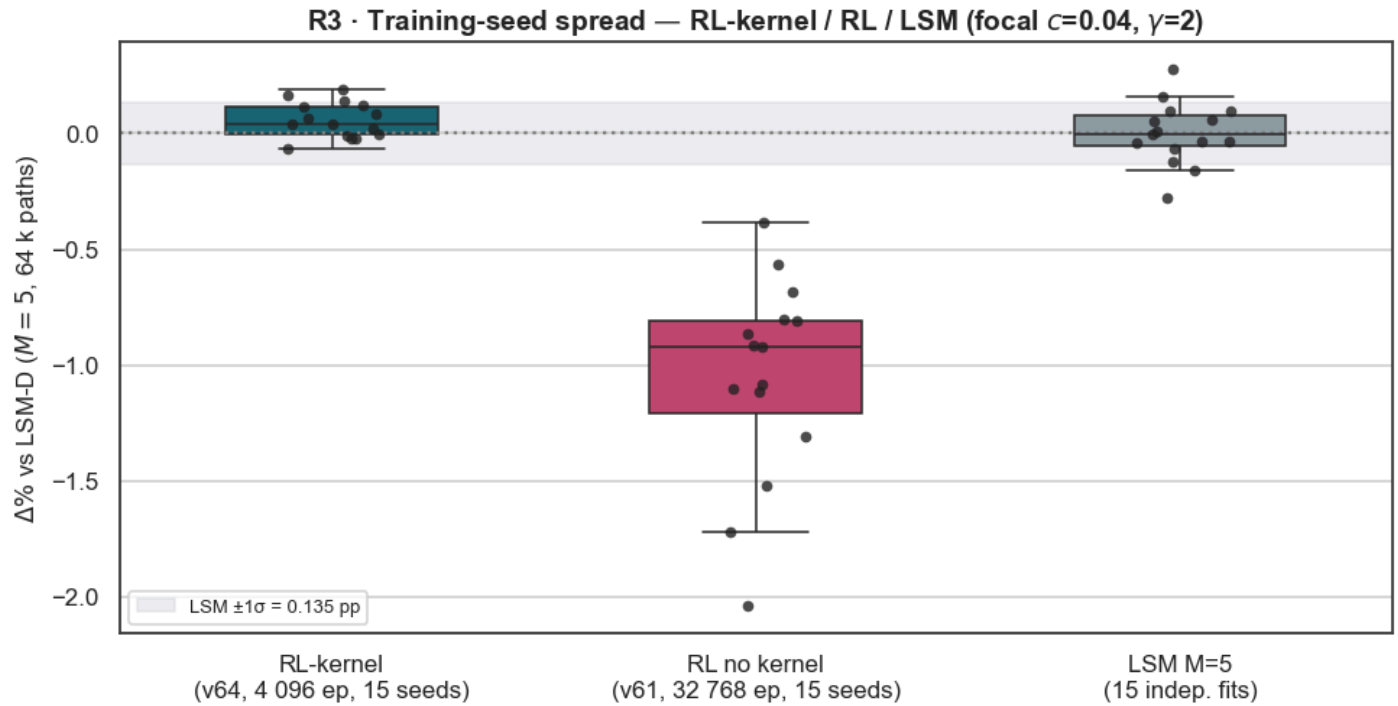
RL-kernel matches the no-kernel 32 768-ep price by ~2048 episodes (16× fewer).

R3 · Training-seed dispersion

Three training-randomness distributions on one axis (15 seeds each, focal $c=0.04$, $\gamma=2$):

Column	What varies
RL-kernel (v64, 4k ep)	training seed
RL no kernel (v61, 32k ep)	training seed
LSM $M=5$ (15 indep. fits)	training-sample randomness

RL-kernel seed std (≈ 0.08 pp) is below the LSM training-sample std (≈ 0.14 pp); the no-kernel agent (≈ 0.44 pp) is markedly worse.



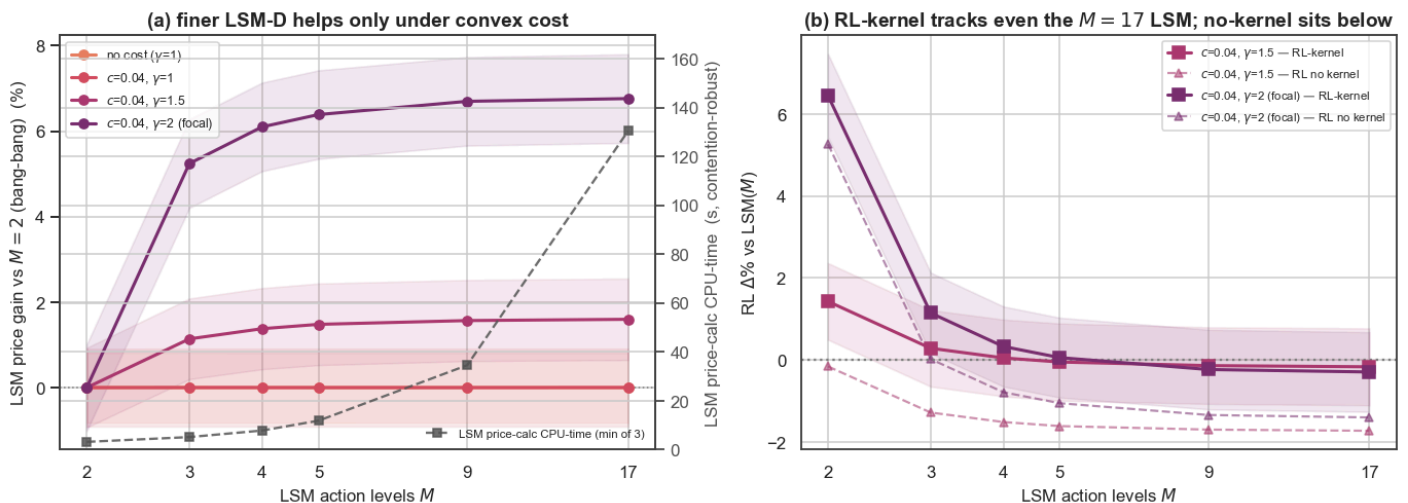
Denominator: LSM-D M=5 (train seed 998 / test seed 999 / 64k paths)

group	mean $\Delta\%$ (pp)	std (pp)	n
RL-kernel (v64, 4k ep)	+0.057	0.076	15
RL no kernel (v61, 32k ep)	-1.056	0.442	15
LSM M=5 (indep. fits)	+0.001	0.135	15

comparison	Δmean (pp)	Welch p	var ratio	F p
kernel vs no-kernel	+1.11	0.000	33.5×	5.1e-08
kernel vs LSM	—	—	3.11×	4.2e-02

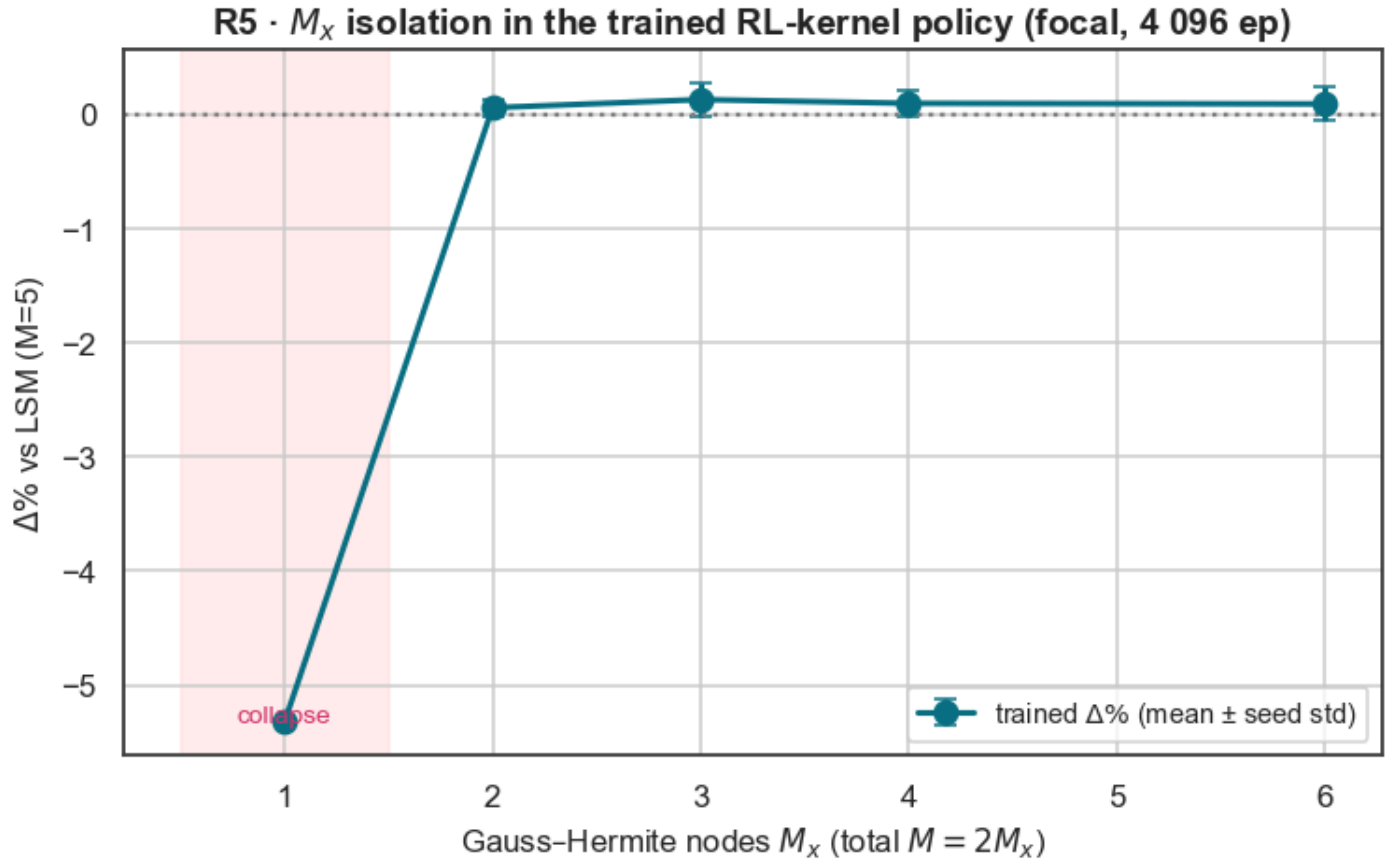
R4 · How much of the RL–LSM gap is the action discretisation?

We sweep the LSM action grid $M \in \{2, 3, 4, 5, 9, 17\}$. **(a)** Discretisation only matters under convex cost ($\gamma > 1$); the LSM price is flat in M for no-cost/ $\gamma=1$. **(b)** RL-kernel tracks even the $M=17$ grid; RL no-kernel sits ≈ 1 –2 pp below at every M — the kernel edge is not a discretisation artefact.



R5 · End-to-end role of M_x

F2 showed the quadrature target plateaus once $M_x \geq 2$. Here we confirm the same threshold in the **trained policy's price**: sweeping $M_x \in \{1, 2, 3, 4, 6\}$ at 4k episodes, $M_x=1$ collapses while $M_x \geq 2$ is statistically flat.



M_x significance tests — $M_x \geq 2$ plateau: all pairwise Welch p > 0.20 (green = n.s.)

comparison	Δ mean (pp)	Welch p
$M_x=2$ vs $M_x=1$ (collapse threshold)	+5.38	0.0e+00
$M_x=2$ vs $M_x=3$ (plateau check)	-0.07	0.50
$M_x=2$ vs $M_x=4$ (plateau check)	-0.04	0.63
$M_x=2$ vs $M_x=6$ (plateau check)	-0.03	0.73
$M_x=3$ vs $M_x=4$ (plateau check)	+0.03	0.78
$M_x=3$ vs $M_x=6$ (plateau check)	+0.04	0.77
$M_x=4$ vs $M_x=6$ (plateau check)	+0.01	0.96

Summary — headline table and hypothesis tests

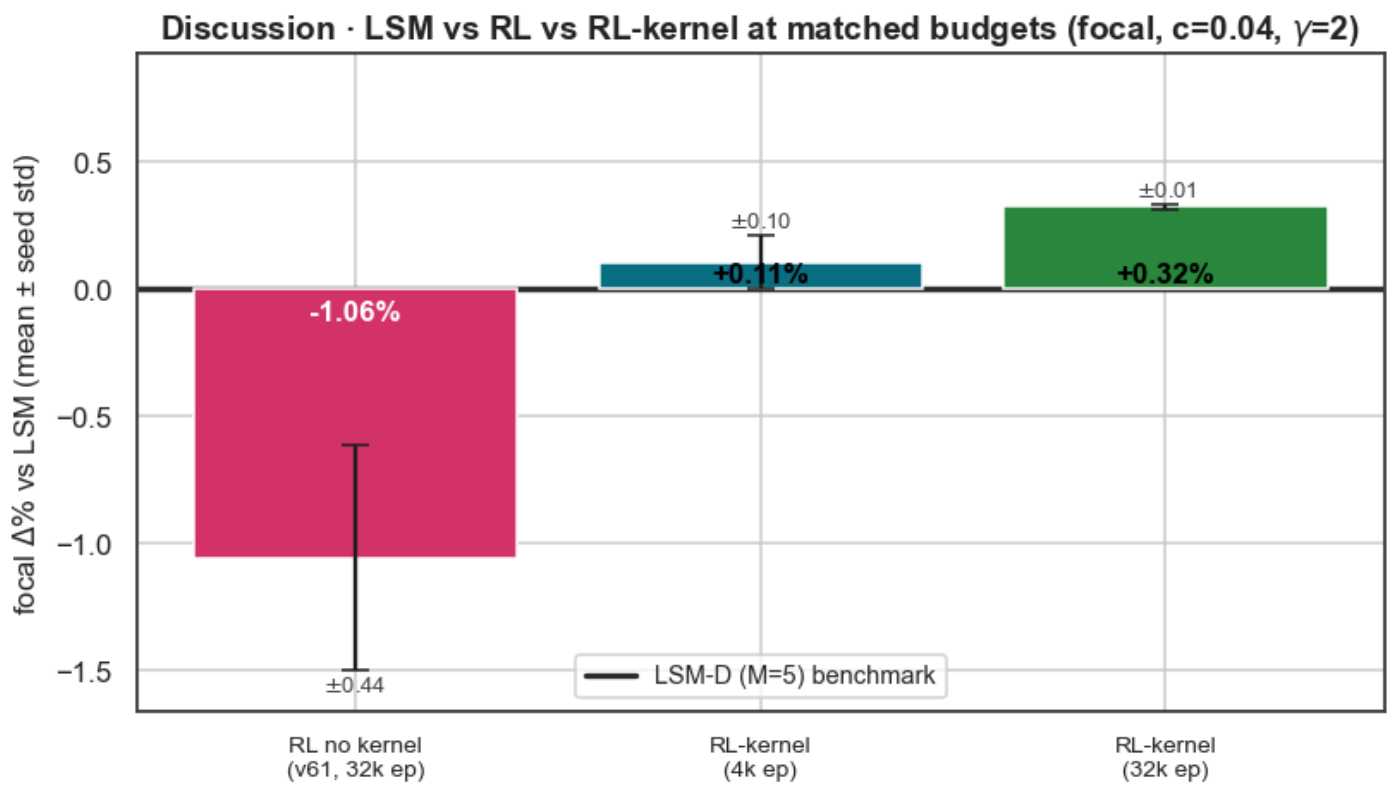
Focal $c=0.04$, $\gamma=2$ — $\Delta\%$ vs full-state LSM-D ($M=5$)

method	episodes	n seeds	mean $\Delta\%$	seed std	best $\Delta\%$	worst $\Delta\%$
RL-kernel (v64)	4,096	15	+0.057	0.076	+0.189	-0.068
RL no kernel (v61)	32,768	15	-1.056	0.442	-0.385	-2.042

stat	value	p
RL-kernel – RL no kernel (Δ mean)	+1.11 pp	0.000
Variance ratio (no-kernel / kernel)	33.5x	5.1e-08
Min detectable effect (80% power, n=15/15)	0.32 pp	—

Discussion · Three-way comparison at matched budgets

Both LSM-D and RL rollout are lower bounds, so the higher OOS price is the better estimate. RL no-kernel at 32k ep sits ≈ 1 pp below LSM-D; RL-kernel reaches parity at 4k and crosses above at 32k. The deterministic expected backup removes the single-sample TD noise that forces heavy regularisation in the no-kernel recipe.



RL no kernel (32k) mean delta: -1.06% | RL-kernel (4k) mean delta: $+0.11\%$
RL-kernel (32k) mean delta: $+0.32\%$

Part II takeaway. RL-kernel prices at or above full-state LSM-D — competitive even with the $M=17$ benchmark — while RL without the kernel sits ≈ 1 pp below at $8\times$ the budget. The kernel's decisive contribution is **efficiency and stability**: LSM parity at 4k episodes, tighter seed dispersion than both alternatives, and the $M_x \geq 2$ threshold from Part I governing the trained outcome end-to-end.

Reproducibility

CSVs produced by `gen_kernel_validation.py` (Part I), `gen_rl_validation.py` (R1–R5), `gen_r4_discretisation.py` (R4). RL-kernel = v64 sweep agents; RL no-kernel = saved v61 prices

(Convex Costs Results 7.csv); reproducible no-kernel config:

tools/train_hedging_agents.py . Test: seed 999, 65 536 paths; LSM-train: seed 998.

Appendix · Full convex-cost grid — RL (no kernel) vs RL-kernel, $\Delta\%$ vs LSM-D

$\Delta\% = 100 (\bar{V}_{\text{RL}}/V_{\text{LSM}} - 1)$ across the full $c \times \gamma$ grid (seeds {11, 12, 13}), evaluated on the common test set (seed 999, 65 536 paths) against full-state LSM-D (Chebyshev deg-2, $M=5$, train seed 998) via tools/rebuild_results_v7.py .

RL no kernel (v61): $\Delta\% \geq 0$ in 3/25 cells | mean $\Delta\% = -0.85$ pp

RL-kernel (v64, 4k): $\Delta\% \geq 0$ in 12/25 cells | mean $\Delta\% = +0.35$ pp

$\Delta\%$ vs full-state LSM-D (Chebyshev deg-2, $M=5$) on the common test set (seed 999, 65 536 paths). Green = RL $\geq$ LSM, red = below; bold = best per (c, γ).

	c	gamma	LSM-D	$\Delta\%$ RL no kernel (v61)	$\Delta\%$ RL-kernel (v64, 4k)
0	0.01	1.0	2.5549	-0.82	-0.18
1	0.01	1.5	2.5131	-0.90	-0.17
2	0.01	2.0	2.4609	-1.16	-0.14
3	0.01	3.0	2.3289	-1.29	+0.01
4	0.02	1.0	2.4452	-1.19	-0.13
5	0.02	1.5	2.3690	-1.17	-0.11
6	0.02	2.0	2.2822	-1.25	-0.07
7	0.02	3.0	2.1030	-1.10	+0.42
8	0.04	1.0	2.2378	-0.78	-0.18
9	0.04	1.5	2.1067	-1.61	-0.05
10	0.04	2.0	1.9822	-1.06	+0.11
11	0.04	3.0	1.7917	-0.08	+1.14
12	0.05	1.0	2.1392	-0.97	-0.17
13	0.05	1.5	1.9869	-1.30	-0.01
14	0.05	2.0	1.8541	-1.33	+0.15
15	0.05	3.0	1.6749	+0.56	+1.54
16	0.08	1.0	1.8632	-1.00	-0.24
17	0.08	1.5	1.6666	-1.77	+0.25
18	0.08	2.0	1.5332	-0.13	+0.80
19	0.10	1.0	1.6949	-0.49	-0.21
20	0.10	1.5	1.4825	-1.00	+0.49
21	0.10	2.0	1.3606	+0.13	+1.30
22	0.15	1.0	1.3295	-1.12	-0.18
23	0.15	1.5	1.1082	-2.01	+1.24
24	0.15	2.0	1.0321	+1.49	+3.11